ADITYA UNIVERSITY

JAVA PROGRAMMING LAB MANUAL

Course Code: 241IT006

—

Semester: IV Semester

—

Regulations : AR24

INDEX

S.No	Name Of The Experiment	Page No
1	Write a Java program which selects and prints all the prime numbers within the range of 1 to 100.	1
2	Write a Java Program which finds the sum of all even terms in the Fibonacci sequence up to the given range N.	2
3	Write a Java program to check whether a given number is Armstrong or not.An Armstrong number (also called a Narcissistic number) is a number that is equal to the sum of its own digits raised to the power of the number of digits.	4
4	Write a Java program to sort an array of integers in ascending order	6
5	write a java program to find the maximum and minimum element in an array	8
6	write a java program to remove the duplicate elements in the array	10
7	Write a Java Program to display the details of a person. Personal details should be given in one method and the qualification details in another method.	12
8	Write a Java Program to implement constructor and constructor overloading.	15
9	Write a Java Program to implement method overloading.	18
10	Write a java program to check whether the given string is pangram or not (contains every letter of the alphabet atleast once)	20
11	Write a java program to find the most frequently occurring character in a string.	22
12	Write a Java Program to find all permutations of a given string.	23
13	Write a Java Program to Check if a given string is a anagram (Ex: CAT and ACT).	24
14	Write a java program implementing multi level Inheritance.	25
15	Write a Java Program to implement multiple Inheritance.	28
16	Write a Java program to find the areas of different shapes using abstract classes.	30

S.No	Name Of The Experiment	Page No
17	Write a Java program to import and use user defined package.	32
18	Write a Java program to illustrate the use of protected members in a package.	33
19	Write a java program to copy Even numbers into Even.txt file and Odd Numbers into Odd.txt file.	34
20	Write a java program to make use of ArrayList and LinkedList	36
21	Write a java program to make use of Iterator and Iterable	38
22	Write a java program to make use of Comparator and Comparable	39
23	Write a java program to make use of HashMap and TreeMap	41
24	Write a java program to make use of HashSet and TreeSet	43
25	Write a java program to make use of HashTable	45
26	Write a Java program to illustrate exception handling mechanism using multiple catch clauses.	47
27	Write a Java program to Make use of Built-in and user-defined Exceptions in handling a run time exception.	49
28	Write a Java program that creates threads by extending Thread class. First thread display “Good Morning “every 1 sec, the second thread displays “Hello “every 2 seconds and the third display “Welcome” every 3 seconds.	52
29	Write a Java program that creates threads by implementing Runnable Interface. First thread display “Good Morning “every 1 sec, the second thread displays “Hello “every 2 seconds and the third display “Welcome” every 3 seconds.	54
30	Write a java program to solve Producer – Concumer problem using synchronization	56
31	Write a Java program to implement CRUD operations on a Database using JDBC API	59

Program 1:

Write a Java program which selects and prints all the prime numbers within the range of 1 to 100.

Program:

PrimeNumbersAppl.java:

```
public class PrimeNumbersAppl {
    public static void main(String[] args) {
        System.out.println("Prime numbers between 1 and 100 are:");
        for (int num = 2; num <= 100; num++) {
            boolean isPrime = true;
            // Check if num is divisible by any number from 2 to num/2
            for (int i = 2; i <= num / 2; i++) {
                if (num % i == 0) {
                    isPrime = false;
                    break;
                }
            }
            // Print num if it is prime
            if (isPrime) {
                System.out.print(num + " ");
            }
        }
    }
}
```

Output:

Compilation: javac PrimeNumbersAppl.java

Execution : java PrimeNumbersAppl

Prime numbers between 1 and 100 are:

2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97

Program 2:

Write a Java Program which finds the sum of all even terms in the Fibonacci sequence up to the given range N.

Program:

EvenFibonacciSum.java:

```
import java.util.Scanner;
public class EvenFibonacciSum {
   public static void main(String[] args) {
      int n,first = 0, second = 1,sum = 0,next;
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter the range : ");
      n = sc.nextInt();
      System.out.println("Fibonacci sequence up to " + n + ":");
      // Print first term if within range
      if(first <= n) {
         System.out.print(first + " ");
      }
      if(second <= n) {
         System.out.print(second + " ");
      }
      while (true) {
         next = first + second;
         if (next > n) {
            break;
         }
         // Print the term
         System.out.print(next + " ");
         // Add even term to sum
         if (next % 2 == 0) {
            sum += next;
         }
```

```
                first = second;
                second = next;
            }
            System.out.println("\nSum of even Fibonacci numbers up to " + n + " = " + sum);
        }
    }
```

Output:

Compilation: javac EvenFibonacciSum.java

Execution : java EvenFibonacciSum

Enter the range : 13

Fibonacci sequence up to 13:

0 1 1 2 3 5 8 13

Sum of even Fibonacci numbers up to 13 = 10

Program 3:

Write a Java program to check whether a given number is Armstrong or not.An Armstrong number (also called a Narcissistic number) is a number that is equal to the sum of its own digits raised to the power of the number of digits.

Program:

ArmstrongNumber.java:

```
import java.util.Scanner;
public class ArmstrongNumber {
   public static void main(String[] args) {
      int num,sum = 0,digits = 0,temp;
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a number: ");
      num = sc.nextInt();
      // Count digits
      temp = num;
      while (temp > 0) {
         digits++;
         temp /= 10;
      }
      // Calculate Armstrong sum
      temp = num;
      while (temp > 0) {
         int remainder = temp % 10;
         sum += Math.pow(remainder, digits);
         temp /= 10;
      }
      // Check condition
      if (sum == num) {
         System.out.println(num + " is an Armstrong number.");
      } else {
         System.out.println(num + " is NOT an Armstrong number.");
      }
```

```
        }
    }
```

Output:

Compilation: javac ArmstrongNumber.java

Execution : java ArmstrongNumber

Enter a number: 153

153 is an Armstrong number.

Execution : java ArmstrongNumber

Enter a number: 1634

1634 is an Armstrong number.

Program 4:

Write a Java program to sort an array of integers in ascending order

Program:

SortingAppl.java:

```
import java.util.Scanner;
public class SortingAppl {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Read array size
        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        // Read array elements
        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        System.out.println("\nGiven order of elements:");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
        // Bubble Sort logic
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - 1 - i; j++) {
                if (arr[j] > arr[j + 1]) {
                    // swap arr[j] and arr[j+1]
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
```

```
        // Display sorted array
        System.out.println("\nSorted array in ascending order:");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```

Output:

Compilation: javac SortingAppl.java

Execution : java SortingAppl

Enter number of elements: 8

Enter array elements:

10

2

92

32

86

34

45

23

Given order of elements:

10 2 92 32 86 34 45 23

Sorted array in ascending order:

2 10 23 32 34 45 86 92

Program 5:

write a java program to find the maximum and minimum element in an array

Program:

MaxMinArray.java:

```
import java.util.Scanner;
public class MaxMinArray {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      // Read array size
      System.out.print("Enter number of elements: ");
      int n = sc.nextInt();
      int[] arr = new int[n];
      // Read array elements
      System.out.println("Enter array elements:");
      for (int i = 0; i < n; i++) {
         arr[i] = sc.nextInt();
      }
      // Display given order of elements
      System.out.println("\nGiven array elements:");
      for (int i = 0; i < n; i++) {
         System.out.print(arr[i] + " ");
      }
      // Initialize max and min with first element
      int max = arr[0];
      int min = arr[0];
      // Find maximum and minimum
      for (int i = 1; i < n; i++) {
         if (arr[i] > max) {
            max = arr[i];
         }
         if (arr[i] < min) {
            min = arr[i];
```

```
            }
        }
        // Display results
        System.out.println("\n\nMaximum element = " + max);
        System.out.println("Minimum element = " + min);
    }
}
```

Output:

Compilation: javac MaxMinArray.java

Execution : java MaxMinArray

Enter number of elements: 5

Enter array elements:

12

3

78

34

45

Given array elements:

12 3 78 34 45

Maximum element = 78

Minimum element = 3

Program 6:

write a java program to remove the duplicate elements in the array

Program:

RemDupArray.java:

```
import java.util.Scanner;
public class RemDupArray {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      // Read array size
      System.out.print("Enter number of elements: ");
      int n = sc.nextInt();
      int[] arr = new int[n];
      int[] brr = new int[n];
      // Read array elements
      System.out.println("Enter array elements:");
      for (int i = 0; i < n; i++) {
         arr[i] = sc.nextInt();
      }
      // Display given order of elements
      System.out.println("\nGiven array elements:");
      for (int i = 0; i < n; i++) {
         System.out.print(arr[i] + " ");
      }
      int newSize = 0;
      for (int i = 0; i < n; i++) {
         boolean isDup = false;
         for (int j = 0; j < newSize; j++) {
            if (arr[i] == brr[j]) {
               isDup = true;
               break;
            }
         }
```

```
            if (!isDup) {
                brr[newSize] = arr[i];
                newSize++;
            }
        }
        System.out.println("\nGiven array elements after removing duplicates:");
        for (int i = 0; i < newSize; i++) {
        System.out.print(brr[i] + " ");
        }
    }
}
```

Output:

Compilation: javac RemDupArray.java

Execution : java RemDupArray

Enter number of elements: 8

Enter array elements:

10

20

30

20

40

10

80

60

Given array elements:

10 20 30 20 40 10 80 60

Given array elements after removing duplicates:

10 20 30 40 80 60

Program 7:

Write a Java Program to display the details of a person. Personal details should be given in one method and the qualification details in another method.

Program:

Person.java:

```
import java.util.Scanner;
class Person{
   String name, gender, city;
   int age;
   String degree, branch, university;
   int year;
  Scanner sc = new Scanner(System.in);
  void readPersonalDetails() {
      System.out.println("Enter Personal Details:");
      System.out.print("Name: ");
      name = sc.nextLine();
      System.out.print("Age: ");
      age = sc.nextInt();
      sc.nextLine();
      System.out.print("Gender: ");
      gender = sc.nextLine();
      System.out.print("City: ");
      city = sc.nextLine();
  }
  void readQualificationDetails() {
      System.out.println("\nEnter Qualification Details:");
      System.out.print("Degree: ");
      degree = sc.nextLine();
      System.out.print("Branch: ");
      branch = sc.nextLine();
      System.out.print("University: ");
      university = sc.nextLine();
```

```
        System.out.print("Year of Passing: ");
        year = sc.nextInt();
    }
    void displayPersonalDetails() {
        System.out.println("\nPersonal Details:");
        System.out.println("Name   : " + name);
        System.out.println("Age    : " + age);
        System.out.println("Gender : " + gender);
        System.out.println("City   : " + city);
    }
    void displayQualificationDetails() {
        System.out.println("\nQualification Details:");
        System.out.println("Degree     : " + degree);
        System.out.println("Branch     : " + branch);
        System.out.println("University : " + university);
        System.out.println("Year       : " + year);
    }
    public static void main(String[] args) {
        Person p = new Person();
        p.readPersonalDetails();
        p.readQualificationDetails();
        p.displayPersonalDetails();
        p.displayQualificationDetails();
    }
}
```

Output:

Compilation: javac Person.java

Execution : java Person

```
Enter Personal Details:
Name: Chakri
Age: 41
Gender: Male
City: Kakinada
```

Enter Qualification Details:

Degree: B.Tech

Branch: CSE

University: JNTUK

Year of Passing: 2006

Personal Details:

Name : Chakri

Age : 41

Gender : Male

City : Kakinada

Qualification Details:

Degree : B.Tech

Branch : CSE

University : JNTUK

Year : 2006

Program 8:

Write a Java Program to implement constructor and constructor overloading.

Program:

Student.java:

```
class Student {
    String name;
    int age;
    String course;
    // Default constructor
    Student() {
        name = "Not Assigned";
        age = 0;
        course = "Not Selected";
    }
    // Constructor with one parameter
    Student(String n) {
        name = n;
        age = 0;
        course = "Not Selected";
    }
    // Constructor with two parameters
    Student(String n, int a) {
        name = n;
        age = a;
        course = "Not Selected";
    }
    // Constructor with three parameters
    Student(String n, int a, String c) {
        name = n;
        age = a;
        course = c;
    }
```

```
        void display() {
            System.out.println("Name   : " + name);
            System.out.println("Age    : " + age);
            System.out.println("Course : " + course);
            System.out.println();
        }
        public static void main(String[] args) {
            Student s1 = new Student();                    // default constructor
            Student s2 = new Student("Ravi");              // one parameter
            Student s3 = new Student("Anita", 20);         // two parameters
            Student s4 = new Student("Kiran", 22, "Java");  // three parameters
            System.out.println("Student details created with default constructor");
            s1.display();
           System.out.println("Student details created with parameterised constructor(String)");
            s2.display();
      System.out.println("Student details created with parameterised constructor(String,int)");
            s3.display();
            System.out.println("Student details created with parameterised
                                                    constructor(String,int,String)");
            s4.display();
        }
    }
```

Output:

Compilation: javac Student.java

Execution : java Student

```
Student details created with default constructor
Name   : Not Assigned
Age    : 0
Course : Not Selected
Student details created with parameterised constructor(String)
Name   : Ravi
```

Age : 0

Course : Not Selected

Student details created with parameterised constructor(String,int)

Name : Anita

Age : 20

Course : Not Selected

Student details created with parameterised constructor(String,int,String)

Name : Kiran

Age : 22

Course : Java

Program 9:

Write a Java Program to implement method overloading.

Program:

MethodOverloadingAppl.java:

```
class MethodOverloadingAppl {
    // Method with two integer parameters
    int add(int a, int b) {
        System.out.println("add(int, int) method is used");
        return a + b;
    }
    // Method with three integer parameters
    int add(int a, int b, int c) {
        System.out.println("add(int, int, int) method is used");
        return a + b + c;
    }
    // Method with two double parameters
    double add(double a, double b) {
        System.out.println("add(double, double) method is used");
        return a + b;
    }
    // Method with int and double parameters
    double add(int a, double b) {
        System.out.println("add(int, double) method is used");
        return a + b;
    }
    // Method with double and int parameters
    double add(double a, int b) {
        System.out.println("add(double, int) method is used");
        return a + b;
    }
```

```
    public static void main(String[] args) {
        MethodOverloadingAppl obj = new MethodOverloadingAppl();
        int sum1 = obj.add(10, 20);
        System.out.println("Returned Sum: " + sum1 + "\n");
        int sum2 = obj.add(5, 10, 15);
        System.out.println("Returned Sum: " + sum2 + "\n");
        double sum3 = obj.add(10.5, 20.3);
        System.out.println("Returned Sum: " + sum3 + "\n");
        double sum4 = obj.add(10, 20.5);
        System.out.println("Returned Sum: " + sum4 + "\n");
        double sum5 = obj.add(15.5, 10);
        System.out.println("Returned Sum: " + sum5);
    }
}
```

Output:

Compilation: javac MethodOverloadingAppl.java

Execution : java MethodOverloadingAppl

```
add(int, int) method is used

Returned Sum: 30

add(int, int, int) method is used

Returned Sum: 30

add(double, double) method is used

Returned Sum: 30.8

add(int, double) method is used

Returned Sum: 30.5

add(double, int) method is used

Returned Sum: 25.5
```

Program 10:

Write a java program to check whether the given string is pangram or not (contains every letter of the alphabet atleast once)

Program:

PangramCheck.java:

```
import java.util.Scanner;
public class PangramCheck {
    public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter a sentence: ");
    String str = sc.nextLine().toLowerCase();
    boolean[] alphabet = new boolean[26];
    int index;
    for (int i = 0; i < str.length(); i++) {
      char ch = str.charAt(i);
      if (ch >= 'a' && ch <= 'z') {
        index = ch - 'a';
        alphabet[index] = true;
      }
    }
    boolean isPangram = true;
    for (int i = 0; i < 26; i++) {
      if (!alphabet[i]) {
        isPangram = false;
        break;
      }
    }
    if (isPangram)
      System.out.println("The given string is a Pangram.");
    else
      System.out.println("The given string is NOT a Pangram.");
```

```
            sc.close();
        }
    }
```

Output:

Compilation: javac PangramCheck.java

Execution : java PangramCheck

Enter a sentence: The quick brown fox jumps over the lazy dog

The given string is a Pangram.

Execution : java PangramCheck

Enter a sentence: hello students how are you

The given string is NOT a Pangram.

Program 11:

Write a java program to find the most frequently occurring character in a string.

Program:

MostFrequentCharacter.java:

```
import java.util.Scanner;
public class MostFrequentCharacter {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String str = sc.nextLine();
        int maxCount = 0;
        char maxChar = ' ';
        for (int i = 0; i < str.length(); i++) {
            int count = 0;
            for (int j = 0; j < str.length(); j++) {
                if (str.charAt(i) == str.charAt(j)) {
                    count++;
                }
            }
            if (count > maxCount) {
                maxCount = count;
                maxChar = str.charAt(i);
            }
        }
        System.out.println("Most frequent character: " + maxChar);
        System.out.println("Frequency: " + maxCount);
        sc.close();
    }
}
```

Output:

Compilation: javac MostFrequentCharacter.java

Execution : java MostFrequentCharacter

```
Enter a string: Java Programming
Most frequent character: a
Frequency: 3
```

Program 12:

Write a Java Program to find all permutations of a given string.

Program:

StringPermutations.java:

```
import java.util.Scanner;
public class StringPermutations {
   // Method to generate permutations
   public static void permute(String str, String result) {
      if (str.length() == 0) {
         System.out.println(result);
         return;
      }
      for (int i = 0; i < str.length(); i++) {
         char ch = str.charAt(i);
         // Remaining string after removing the selected character
         String remaining = str.substring(0, i) + str.substring(i + 1);
         permute(remaining, result + ch);
      }
   }
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a string: ");
      String str = sc.nextLine();
      System.out.println("Permutations of the string are:");
      permute(str, "");
      sc.close();
   }
}
```

Output:

Compilation: javac StringPermutations.java

Execution : java StringPermutations

```
Enter a string: abc
Permutations of the string are:
abc
acb
bac
bca
cab
cba
```

Program 13:

Write a Java Program to Check if a given string is a anagram (Ex: CAT and ACT).

Program:

AnagramCheck.java:

```
import java.util.Arrays;
import java.util.Scanner;
public class AnagramCheck {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter first string: ");
      String str1 = sc.nextLine();
      System.out.print("Enter second string: ");
      String str2 = sc.nextLine();
      // Convert strings to lowercase
      str1 = str1.toLowerCase();
      str2 = str2.toLowerCase();
      // Convert strings to character arrays
      char[] arr1 = str1.toCharArray();
      char[] arr2 = str2.toCharArray();
      // Sort both arrays
      Arrays.sort(arr1);
      Arrays.sort(arr2);
      // Compare arrays
      if (Arrays.equals(arr1, arr2))
         System.out.println("The given strings are Anagrams.");
      else
         System.out.println("The given strings are NOT Anagrams.");
      sc.close();
   }
}
```

Output:

Compilation: javac AnagramCheck.java

Execution : java AnagramCheck

```
Enter first string: LISTEN
Enter second string: SILENT
The given strings are Anagrams.
```

Execution : java AnagramCheck

```
Enter first string: CAT
Enter second string: MAT
The given strings are NOT Anagrams.
```

Program 14:

Write a java program implementing multi level Inheritance.

Program:

MultilevelInheritanceDemo.java:

```
class Person {
    protected String name;
    protected int age;
    // Constructor
    Person(String name, int age) {
        this.name = name;
        this.age = age;
        System.out.println("Person constructor called");
    }
    // Method
    public void displayDetails() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
    // final method (cannot be overridden)
    public final void showCategory() {
        System.out.println("Category: Human");
    }
}
class Employee extends Person {
    protected int empId;
    protected double salary;
    Employee(String name, int age, int empId, double salary) {
        super(name, age);  // calling parent constructor
        this.empId = empId;
        this.salary = salary;
        System.out.println("Employee constructor called");
    }
```

```
    // Method overriding
    public void displayDetails() {
        super.displayDetails();  // calling parent method
        System.out.println("Employee ID: " + empId);
        System.out.println("Salary: " + salary);
    }
    public void work() {
        System.out.println(name + " is working as an employee");
    }
}
class Manager extends Employee {
    private String department;
    Manager(String name, int age, int empId, double salary, String department) {
        super(name, age, empId, salary);  // calling Employee constructor
        this.department = department;
        System.out.println("Manager constructor called");
    }
    // Method overriding
    public void displayDetails() {
        super.displayDetails();  // calling Employee version
        System.out.println("Department: " + department);
    }
    public void manageTeam() {
        System.out.println(name + " is managing " + department + " department");
    }
}
public class MultilevelInheritanceDemo {
    public static void main(String[] args) {
        // Creating Manager object
        Manager m = new Manager("Kiran", 35, 101, 75000, "IT");
        System.out.println("\n--- Display Details ---");
        m.displayDetails();
```

```
        System.out.println("\n--- Calling methods ---");
        m.work();
        m.manageTeam();
        m.showCategory();
        System.out.println("\n--- Dynamic Method Dispatch ---");
        Person p = new Manager("Ravi", 40, 102, 90000, "HR");
        p.displayDetails();  // runtime polymorphism
    }
}
```

Output:

Compilation: javac MultilevelInheritanceDemo.java

Execution : java MultilevelInheritanceDemo

```
Person constructor called
Employee constructor called
Manager constructor called
--- Display Details ---
Name: Kiran
Age: 35
Employee ID: 101
Salary: 75000.0
Department: IT
--- Calling methods ---
Kiran is working as an employee
Kiran is managing IT department
Category: Human

--- Dynamic Method Dispatch ---
Person constructor called
Employee constructor called
Manager constructor called
Name: Ravi
Age: 40
Employee ID: 102
Salary: 90000.0
Department: HR
```

Program 15:

Write a Java Program to implement multiple Inheritance.

Program:

MultipleInheritanceDemo.java:

```
// First interface
interface Teacher {
   int hours = 5;  // public static final by default
   void teach();   // public abstract by default
   }
// Second interface
interface Researcher {
   int papers = 10;
   void doResearch();
   }
// Class implementing multiple interfaces
class Professor implements Teacher, Researcher {
    String name;
    // Constructor
   Professor(String name) {
      this.name = name;
   }
   // Implement Teacher method
   public void teach() {
      System.out.println(name + " teaches for " + hours + " hours per day");
   }
   // Implement Researcher method
   public void doResearch() {
      System.out.println(name + " publishes " + papers + " research papers");
   }
   // Own method
   public void display() {
      System.out.println("Professor Name: " + name);
```

```
        }
    }
    // Main class
    public class MultipleInheritanceDemo {
        public static void main(String[] args) {
        Professor p = new Professor("Kiran");
        p.display();
        p.teach();
        p.doResearch();
        }
    }
```

Output:

Compilation: javac MultipleInheritanceDemo.java

Execution : java MultipleInheritanceDemo

Professor Name: Kiran

Kiran teaches for 5 hours per day

Kiran publishes 10 research papers

Program 16:

Write a Java program to find the areas of different shapes using abstract classes.

Program:

AbstractShapeDemo.java:

```
// Abstract class
abstract class Shape {
    // Abstract method (no body)
  abstract void calculateArea();
}
// Circle class
class Circle extends Shape {
  double radius;
   // Constructor
  Circle(double radius) {
    this.radius = radius;
  }
  // Implement abstract method
  void calculateArea() {
    double area = Math.PI * radius * radius;
    System.out.println("Area of Circle = " + area);
  }
}
// Rectangle class
class Rectangle extends Shape {
 double length, width;
 // Constructor
  Rectangle(double length, double width) {
    this.length = length;
    this.width = width;
  }
  // Implement abstract method
  void calculateArea() {
```

```
            double area = length * width;
            System.out.println("Area of Rectangle = " + area);
        }
    }
    // Triangle class
    class Triangle extends Shape {
        double base, height;
       // Constructor
        Triangle(double base, double height) {
            this.base = base;
            this.height = height;
        }
        // Implement abstract method
        void calculateArea() {
            double area = 0.5 * base * height;
            System.out.println("Area of Triangle = " + area);
        }
    }
    // Main class
    public class AbstractShapeDemo {
        public static void main(String[] args) {
        // Using runtime polymorphism
            Shape s;
            s = new Circle(5);
            s.calculateArea();
            s = new Rectangle(4, 6);
            s.calculateArea();
            s = new Triangle(3, 8);
            s.calculateArea();
        }
    }
```

Output:

Compilation: javac AbstractShapeDemo.java

Execution : java AbstractShapeDemo

```
Area of Circle = 78.53981633974483
Area of Rectangle = 24.0
Area of Triangle = 12.0
```

Program 17:

Write a Java program to import and use user defined package.

Program:

Calculator.java:

```
package pack1;   // package declaration
public class Calculator {
   public int add(int a, int b) {
      return a + b;
   }
   public int sub(int a, int b) {
      return a - b;
   }
}
```

Test.java:

```
package pack2;
import pack1.Calculator;
class Test
{
        public static void main(String args[])
        {
                Calculator c=new Calculator();
                System.out.println("addition is:" + c.add(10,20));
                System.out.println("Substraction is:" + c.sub(20,10));
        }
}
```

Output:

Compilation: javac -d . Calculator.java
Compilation: javac -d . Test.java
Execution : java pack2.Test

```
addition is:30

Substraction is:10
```

Program 18:

Write a Java program to illustrate the use of protected members in a package.

Program:

Base.java:

```
package pack1;
public class Base {
   protected int num = 50;
   protected void display() {
      System.out.println("Protected number: " + num);
   }
}
```

Derived.java:

```
package pack2;
import pack1.Base;
public class Derived extends Base {
   public static void main(String[] args) {
      Derived obj = new Derived();
      // Accessing protected member through inheritance
      System.out.println("Value of num: " + obj.num);
      obj.display();
   }
}
```

Output:

Compilation: javac -d . *.java

Execution : java pack2.Derived

Value of num: 50

Protected number: 50

Program 19:

Write a java program to copy Even numbers into Even.txt file and Odd Numbers into Odd.txt file.

Program:

EvenOddFileCopy.java:

```
import java.io.*;
import java.util.*;
public class EvenOddFileCopy {
    public static void main(String[] args) {
        try {
            // Input file containing numbers
            File inputFile = new File("Numbers.txt");
            Scanner sc = new Scanner(inputFile);
            // Writers for even and odd files
            FileWriter evenWriter = new FileWriter("Even.txt");
            FileWriter oddWriter = new FileWriter("Odd.txt");
            while (sc.hasNextInt()) {
                int num = sc.nextInt();
                if (num % 2 == 0) {
                    evenWriter.write(num + " ");
                } else {
                    oddWriter.write(num + " ");
                }
            }
            sc.close();
            evenWriter.close();
            oddWriter.close();
            System.out.println("Numbers copied successfully!");
        } catch (Exception e) {
            System.out.println(e);
        }
    }
```

```
}
```

Output:

Compilation: javac EvenOddFileCopy.java

Execution : java EvenOddFileCopy

Numbers copied successfully!

Numbers.txt:

1
2
3
4
5
6
7
8
9
10

Even.txt:

2 4 6 8 10

Odd.txt:

1 3 5 7 9

Program 20:

Write a java program to make use of ArrayList and LinkedList

Program:

ListExample.java:

```
import java.util.ArrayList;
import java.util.LinkedList;
public class ListExample {
    public static void main(String[] args) {
        // Creating an ArrayList
        ArrayList<String> arrayList = new ArrayList<>();
        // Adding elements to ArrayList
        arrayList.add("Apple");
        arrayList.add("Banana");
        arrayList.add("Mango");
        arrayList.add("Orange");
        System.out.println("Elements in ArrayList:");
        for (String fruit : arrayList) {
            System.out.println(fruit);
        }
        // Removing an element
        arrayList.remove("Banana");
        System.out.println("ArrayList after removing Banana: " + arrayList);
        // Creating a LinkedList
        LinkedList<String> linkedList = new LinkedList<>();
        // Adding elements to LinkedList
        linkedList.add("Red");
        linkedList.add("Green");
        linkedList.add("Blue");
        linkedList.add("Yellow");
        System.out.println("\nElements in LinkedList:");
        for (String color : linkedList) {
            System.out.println(color);
```

```
        }
        // Adding element at first position
        linkedList.addFirst("Black");

        // Removing last element
        linkedList.removeLast();

        System.out.println("LinkedList after modifications: " + linkedList);
    }
}
```

Output:

Compilation: javac ListExample.java

Execution : java ListExample

```
Elements in ArrayList:
Apple
Banana
Mango
Orange
ArrayList after removing Banana: [Apple, Mango, Orange]

Elements in LinkedList:
Red
Green
Blue
Yellow
LinkedList after modifications: [Black, Red, Green, Blue]
```

Program 21:

Write a java program to make use of Iterator and Iterable

Program:

IteratorExample.java:

```
import java.util.ArrayList;
import java.util.Iterator;
public class IteratorExample {
    public static void main(String[] args) {
        // Creating an ArrayList (Collection implementing Iterable)
        ArrayList<String> names = new ArrayList<>();
        // Adding elements
        names.add("Ravi");
        names.add("Anil");
        names.add("Kiran");
        names.add("Sita");
        // Getting the iterator
        Iterator<String> it = names.iterator();
        System.out.println("Elements in the list:");
        // Traversing the list using Iterator
        while(it.hasNext()) {
            String name = it.next();
            System.out.println(name);
        }
    }
}
```

Output:

Compilation: javac IteratorExample.java

Execution : java IteratorExample

```
Elements in the list:
Ravi
Anil
Kiran
Sita
```

Program 22:

Write a java program to make use of Comparator and Comparable

Program:

ComparatorComparableExample.java:

```
import java.util.*;
// Student class implementing Comparable
class Student implements Comparable<Student> {
   int id;
   String name;
   Student(int id, String name) {
      this.id = id;
      this.name = name;
   }
   // Comparable method - sort by ID
   public int compareTo(Student s) {
      return this.id - s.id;
   }
}
// Comparator class to sort by Name
class NameComparator implements Comparator<Student> {
   public int compare(Student s1, Student s2) {
      return s1.name.compareTo(s2.name);
   }
}
public class ComparatorComparableExample {
   public static void main(String[] args) {
      ArrayList<Student> list = new ArrayList<>();
      list.add(new Student(101, "Ravi"));
      list.add(new Student(102, "Kiran"));
      list.add(new Student(103, "Anil"));
      // Sorting using Comparable (by ID)
      Collections.sort(list);
```

```
            System.out.println("Sorting by ID (Comparable):");
            for (Student s : list) {
                System.out.println(s.id + " " + s.name);
            }
            // Sorting using Comparator (by Name)
            Collections.sort(list, new NameComparator());
            System.out.println("\nSorting by Name (Comparator):");
            for (Student s : list) {
                System.out.println(s.id + " " + s.name);
            }
        }
    }
```

Output:

Compilation: javac ComparatorComparableExample.java

Execution : java ComparatorComparableExample

```
Sorting by ID (Comparable):
101 Ravi
102 Kiran
103 Anil
Sorting by Name (Comparator):
103 Anil
102 Kiran
101 Ravi
```

Program 23:

Write a java program to make use of HashMap and TreeMap

Program:

MapExample.java:

```
import java.util.*;
public class MapExample {
    public static void main(String[] args) {
        // HashMap example
        HashMap<Integer, String> hmap = new HashMap<>();
        hmap.put(45, "Ravi");
        hmap.put(12, "Anil");
        hmap.put(78, "Kiran");
        hmap.put(3, "Sita");
        hmap.put(90, "Ram");
        hmap.put(17, "Latha");
        System.out.println("HashMap Output (No ordering):");
        for (Map.Entry<Integer, String> entry : hmap.entrySet()) {
            System.out.println(entry.getKey() + " : " + entry.getValue());
        }
        // TreeMap example
        TreeMap<Integer, String> tmap = new TreeMap<>();
        tmap.put(45, "Ravi");
        tmap.put(12, "Anil");
        tmap.put(78, "Kiran");
        tmap.put(3, "Sita");
        tmap.put(90, "Ram");
        tmap.put(17, "Latha");
        System.out.println("\nTreeMap Output (Sorted by keys):");
        for (Map.Entry<Integer, String> entry : tmap.entrySet()) {
            System.out.println(entry.getKey() + " : " + entry.getValue());
        }
    }
```

```
}
```

Output:

Compilation: javac MapExample.java

Execution : java MapExample

HashMap Output (No ordering):

17 : Latha

3 : Sita

90 : Ram

12 : Anil

45 : Ravi

78 : Kiran

TreeMap Output (Sorted by keys):

3 : Sita

12 : Anil

17 : Latha

45 : Ravi

78 : Kiran

90 : Ram

Program 24:

Write a java program to make use of HashSet and TreeSet

Program:

SetExample.java:

```
import java.util.HashSet;
import java.util.TreeSet;
public class SetExample {
   public static void main(String[] args) {
      // Creating HashSet
      HashSet<Integer> hset = new HashSet<>();
      hset.add(45);
      hset.add(12);
      hset.add(78);
      hset.add(3);
      hset.add(90);
      hset.add(17);
      System.out.println("HashSet Output (No ordering):");
      for(Integer num : hset)
      {
         System.out.print(num + " ");
      }
      System.out.println("\n");
      // Creating TreeSet
      TreeSet<Integer> tset = new TreeSet<>();
      tset.add(45);
      tset.add(12);
      tset.add(78);
      tset.add(3);
      tset.add(90);
      tset.add(17);
      System.out.println("TreeSet Output (Sorted order):");
      for(Integer num : tset)
```

```
            {
                System.out.print(num + " ");
            }
        }
    }
```

Output:

Compilation: javac SetExample.java

Execution : java SetExample

HashSet Output (No ordering):

17 3 90 12 45 78

TreeSet Output (Sorted order):

3 12 17 45 78 90

Program 25:

Write a java program to make use of HashTable

Program:

HashTableExample.java:

```
import java.util.Hashtable;
import java.util.Map;
public class HashTableExample {
    public static void main(String[] args) {
        // Creating a Hashtable
        Hashtable<Integer, String> ht = new Hashtable<>();
        // Adding key-value pairs
        ht.put(101, "Ravi");
        ht.put(102, "Anil");
        ht.put(103, "Kiran");
        ht.put(104, "Sita");
        System.out.println("Elements in Hashtable:");
        // Traversing Hashtable using Map.Entry
        for (Map.Entry<Integer, String> entry : ht.entrySet()) {
            System.out.println(entry.getKey() + " : " + entry.getValue());
        }
        // Checking a key
        if (ht.containsKey(102)) {
            System.out.println("\nKey 102 exists in Hashtable");
        }
        // Removing an element
        ht.remove(103);
        System.out.println("\nHashtable after removing key 103:");
        for (Map.Entry<Integer, String> entry : ht.entrySet()) {
            System.out.println(entry.getKey() + " : " + entry.getValue());
        }
    }
}
```

Output:

Compilation: javac HashTableExample.java

Execution : java HashTableExample

```
Elements in Hashtable:

104 : Sita

103 : Kiran

102 : Anil

101 : Ravi

Key 102 exists in Hashtable

Hashtable after removing key 103:

104 : Sita

102 : Anil

101 : Ravi
```

Program 26:

Write a Java program to illustrate exception handling mechanism using multiple catch clauses.

Program:

MultipleCatchDemo.java:

```
import java.util.Scanner;
public class MultipleCatchDemo {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      double arr[] = new double[5];
      try {
         // Division part
         System.out.println("Enter first number:");
         int a = sc.nextInt();
         System.out.println("Enter second number:");
         int b = sc.nextInt();
         int result = a / b;   // ArithmeticException possible
         System.out.println("Result = " + result);
         // Array access part
         System.out.println("Enter array index where you want to store the above result:");
         int index = sc.nextInt();
         arr[index]=result;
         System.out.println("Result: "+ result +" stored at index: " + index);
         // ArrayIndexOutOfBoundsException possible
      }
      catch (ArithmeticException e) {
         System.out.println("Error: Cannot divide by zero.");
      }
      catch (ArrayIndexOutOfBoundsException e) {
         System.out.println("Error: Invalid array index entered.");
      }
      catch (Exception e) {
         System.out.println("General Exception Occurred.");
```

```
            }
            finally {
                System.out.println("in finally block");
            }
            sc.close();
            System.out.println("program execution completed");
        }
    }
```

Output:

Compilation: javac MultipleCatchDemo.java

Execution : java MultipleCatchDemo

```
Enter first number:
10
Enter second number:
0
Error: Cannot divide by zero.
in finally block
program execution completed
```

Execution : java MultipleCatchDemo

```
Enter first number:
10
Enter second number:
2
Result = 5
Enter array index where you want to store the above result:
7
Error: Invalid array index entered.
in finally block
program execution completed
```

Execution : java MultipleCatchDemo

```
Enter first number:
10
Enter second number:
2
Result = 5
Enter array index where you want to store the above result:
2
Result: 5 stored at index: 2
in finally block
program execution completed
```

Program 27:

Write a Java program to Make use of Built-in and user-defined Exceptions in handling a run time exception.

Program:

BankApplication.java:

```
import java.util.Scanner;
// User-defined Exception
class InsufficientFundsException extends Exception {
   public InsufficientFundsException(String message) {
      super(message);
   }
}
// Bank Account Class
class BankAccount {
   double balance;
   BankAccount(double balance) {
      this.balance = balance;
   }
   void withdraw(double amount)throws InsufficientFundsException  {
      if (amount > balance) {
         throw new InsufficientFundsException("Insufficient balance in account.");
      } else {
         balance = balance - amount;
         System.out.println("Withdrawal successful.");
         System.out.println("Remaining Balance = " + balance);


      }
   }
}
// Main Class
public class BankApplication {
   public static void main(String[] args) {
```

```
Scanner sc = new Scanner(System.in);
boolean txStatus=false;
try {
   // Taking input as String (for NumberFormatException)
   System.out.println("Enter initial balance:");
   String balInput = sc.nextLine();
   double balance = Double.parseDouble(balInput);
   BankAccount account = new BankAccount(balance);
   System.out.println("Enter amount to withdraw:");
   String amtInput = sc.nextLine();
   double amount = Double.parseDouble(amtInput);
   account.withdraw(amount);
    txStatus=true;
}
// User-defined Exception
catch (InsufficientFundsException e) {
   System.out.println("User Defined Exception: " + e.getMessage());
}
// Built-in Exception
catch (NumberFormatException e) {
   System.out.println("Built-in Exception: Invalid numeric input.");
}
// General Exception
catch (Exception e) {
   System.out.println("Some other exception occurred.");
}
finally {
   if(txStatus)
                    System.out.println("Bank transaction completed Successfully");
   else
            System.out.println("Bank transaction was Unsuccessfull");
}
sc.close();
```

```
        }
    }
```

Output:

Compilation: javac BankApplication.java

Execution : java BankApplication

```
Enter initial balance:
5000
Enter amount to withdraw:
7000
User Defined Exception: Insufficient balance in account.
Bank transaction was Unsuccessfull
```

Execution : java BankApplication

```
Enter initial balance:
5000
Enter amount to withdraw:
2000aa
Built-in Exception: Invalid numeric input.
Bank transaction was Unsuccessfull
```

Execution : java BankApplication

```
Enter initial balance:
5000
Enter amount to withdraw:
3000
Withdrawal successful.
Remaining Balance = 2000.0
Bank transaction completed Successfully
```

Program 28:

Write a Java program that creates threads by extending Thread class. First thread display "Good Morning "every 1 sec, the second thread displays "Hello "every 2 seconds and the third display "Welcome" every 3 seconds.

Program:

ThreadExample.java:

```
class GoodMorningThread extends Thread {
   public void run() {
      try {
         while (true) {
            System.out.println("Good Morning");
            Thread.sleep(1000); // 1 second
         }
      } catch (InterruptedException e) {
         System.out.println(e);
      }
   }
}
class HelloThread extends Thread {
   public void run() {
      try {
         while (true) {
            System.out.println("Hello");
            Thread.sleep(2000); // 2 seconds
         }
      } catch (InterruptedException e) {
         System.out.println(e);
      }
   }
}
class WelcomeThread extends Thread {
   public void run() {
```

```
            try {
                while (true) {
                    System.out.println("Welcome");
                    Thread.sleep(3000); // 3 seconds
                }
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
    public class ThreadExample {
        public static void main(String[] args) {
            GoodMorningThread t1 = new GoodMorningThread();
            HelloThread t2 = new HelloThread();
            WelcomeThread t3 = new WelcomeThread();
            t1.start();
            t2.start();
            t3.start();
        }
    }
```

Output:

Compilation: javac ThreadExample.java

Execution : java ThreadExample

```
Good Morning
Welcome
Hello
Good Morning
Hello
Good Morning
Welcome
Good Morning
Hello
Good Morning
Good Morning
Hello
^C
```

Program 29:

Write a Java program that creates threads by implementing Runnable Interface. First thread display "Good Morning "every 1 sec, the second thread displays "Hello "every 2 seconds and the third display "Welcome" every 3 seconds.

Program:

RunnableExample.java:

```
class GoodMorningRunnable implements Runnable {
   public void run() {
      try {
         while (true) {
            System.out.println("Good Morning");
            Thread.sleep(1000); // 1 second
         }
      } catch (InterruptedException e) {
         System.out.println(e);
      }
   }
}
class HelloRunnable implements Runnable {
   public void run() {
      try {
         while (true) {
            System.out.println("Hello");
            Thread.sleep(2000); // 2 seconds
         }
      } catch (InterruptedException e) {
         System.out.println(e);
      }
   }
}
class WelcomeRunnable implements Runnable {
   public void run() {
```

```
            try {
                while (true) {
                    System.out.println("Welcome");
                    Thread.sleep(3000); // 3 seconds
                }
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
    public class RunnableExample {
        public static void main(String[] args) {
          GoodMorningRunnable gmr=new GoodMorningRunnable();
           HelloRunnable hr=new HelloRunnable();
            WelcomeRunnable wr=new WelcomeRunnable();
            Thread t1 = new Thread(gmr);
            Thread t2 = new Thread(hr);
            Thread t3 = new Thread(wr);
            t1.start();
            t2.start();
            t3.start();
        }
    }
```

Output:

Compilation: javac RunnableExample.java

Execution : java RunnableExample

```
Good Morning
Welcome
Hello
Good Morning
Hello
Good Morning
Welcome
Good Morning
Hello
^C
```

Program 30:

Write a java program to solve Producer – Concumer problem using synchronization

Program:

ProducerConsumerExample.java:

```
class Q {
    int n;
    boolean valueSet = false;
    synchronized int get() {
        while (!valueSet) {
            try {
                wait();
            } catch (InterruptedException e) {
                System.out.println("InterruptedException caught");
            }
        }
        System.out.println("Got: " + n);
        valueSet = false;
        notify();
        return n;
    }
    synchronized void put(int n) {
        while (valueSet) {
            try {
                wait();
            } catch (InterruptedException e) {
                System.out.println("InterruptedException caught");
            }
        }
        this.n = n;
        valueSet = true;
        System.out.println("Put: " + n);
        notify();
```

```
    }
}
class Producer implements Runnable {
    Q q;

    Producer(Q q) {
        this.q = q;
        new Thread(this, "Producer").start();
    }
    public void run() {
        int i = 0;
        while (true) {
            q.put(i++);
        }
    }
}
class Consumer implements Runnable {
    Q q;
    Consumer(Q q) {
        this.q = q;
        new Thread(this, "Consumer").start();
    }
    public void run() {
        while (true) {
            q.get();
        }
    }
}
class ProducerConsumerExample {
    public static void main(String args[]) {
        Q q = new Q();
        new Producer(q);
        new Consumer(q);
```

```
            System.out.println("Press Control-C to stop.");
        }
    }
```

Output:

Compilation: javac ProducerConsumerExample.java

Execution : java ProducerConsumerExample

Put: 0

```
Got: 0
Put: 1
Got: 1
Put: 2
Got: 2
Put: 3
Got: 3
Put: 4
Got: 4
Put: 5
Got: 5
Put: 6
Got: 6
^C
```

Program 31:

Write a Java program to implement CRUD operations on a Database using JDBC API

Program:

AusEmpCRUD.java:

```
import java.sql.*;
import java.util.Scanner;
public class AusEmpCRUD {
        static String url = "jdbc:oracle:thin:@localhost:1521/XE";
                                                // value changes based upon the database
        static String username = "SYSTEM";// value changes based upon the database
        static String password = "Password123";      // value changes based upon the database
   public static void createTable(Connection conn) {
      try {
         Statement stmt = conn.createStatement();
         String sql = "CREATE TABLE ausemp (empno NUMBER(5), empname
VARCHAR2(30), salary NUMBER(10,2))";
         stmt.executeUpdate(sql);
         System.out.println("Table 'ausemp' created successfully.");
      } catch (SQLException e) {
         System.out.println("Table already exists.");
      }
   }
   public static void insertRecord(Connection conn, Scanner sc) {
      try {
         System.out.print("Enter Emp No: ");
         int eno = sc.nextInt();
         sc.nextLine();
         System.out.print("Enter Emp Name: ");
         String ename = sc.nextLine();
         System.out.print("Enter Salary: ");
         double sal = sc.nextDouble();
         PreparedStatement ps = conn.prepareStatement(
```

```
            "INSERT INTO ausemp VALUES(?,?,?)");
        ps.setInt(1, eno);
        ps.setString(2, ename);
        ps.setDouble(3, sal);
        int result = ps.executeUpdate();
        if (result == 1)
            System.out.println("Employee inserted successfully.");
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
public static void displayRecords(Connection conn) {
    try {
        PreparedStatement ps = conn.prepareStatement("SELECT * FROM ausemp");
        ResultSet rs = ps.executeQuery();
        System.out.println("----------------------------------");
        System.out.println("EmpNo\tEmpName\t\tSalary");
        System.out.println("----------------------------------");
        while (rs.next()) {
            System.out.println(
                    rs.getInt(1) + "\t" +
                    rs.getString(2) + "\t\t" +
                    rs.getDouble(3));
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
public static void updateRecord(Connection conn, Scanner sc) {
    try {
        System.out.print("Enter Emp No to update salary: ");
        int eno = sc.nextInt();
        System.out.print("Enter New Salary: ");
```

```
        double sal = sc.nextDouble();
        PreparedStatement ps = conn.prepareStatement(
                "UPDATE ausemp SET salary=? WHERE empno=?");
        ps.setDouble(1, sal);
        ps.setInt(2, eno);
        int result = ps.executeUpdate();
        if (result == 1)
            System.out.println("Salary updated successfully.");
        else
            System.out.println("Employee not found.");
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
public static void deleteRecord(Connection conn, Scanner sc) {
    try {
        System.out.print("Enter Emp No to delete: ");
        int eno = sc.nextInt();
        PreparedStatement ps = conn.prepareStatement(
                "DELETE FROM ausemp WHERE empno=?");
        ps.setInt(1, eno);
        int result = ps.executeUpdate();
        if (result == 1)
            System.out.println("Employee deleted successfully.");
        else
            System.out.println("Employee not found.");
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    try {
```

```
        Class.forName("oracle.jdbc.driver.OracleDriver");
        Connection conn = DriverManager.getConnection(url, username, password);
        System.out.println("Connected to Oracle Database");
        // Step 1: Create table
        createTable(conn);
        int choice;
        do {
            System.out.println("\n===== AUSEMP TABLE MENU =====");
            System.out.println("1. Insert Employee");
            System.out.println("2. Update Employee Salary");
            System.out.println("3. Display Employees");
            System.out.println("4. Delete Employee");
            System.out.println("5. Exit");
            System.out.print("Enter choice: ");
            choice = sc.nextInt();
            switch (choice) {
                case 1:
                    insertRecord(conn, sc);
                    break;
                case 2:
                    updateRecord(conn, sc);
                    break;
                case 3:
                    displayRecords(conn);
                    break;
                case 4:
                    deleteRecord(conn, sc);
                    break;
                case 5:
                    System.out.println("Exiting program...");
                    break;
                default:
                    System.out.println("Invalid choice.");
```

```
            }

        } while (choice != 5);
        conn.close();
    } catch (Exception e) {
        e.printStackTrace();
    }
  }
}
```

Output:

Compilation: javac AusEmpCRUD.java

Execution : java AusEmpCRUD

```
Connected to Oracle Database
Table 'ausemp' created successfully.
===== AUSEMP TABLE MENU =====
1. Insert Employee
2. Update Employee Salary
3. Display Employees
4. Delete Employee
5. Exit
Enter choice: 1
Enter Emp No: 10
Enter Emp Name: chakri
Enter Salary: 70000
Employee inserted successfully.
===== AUSEMP TABLE MENU =====
1. Insert Employee
2. Update Employee Salary
3. Display Employees
4. Delete Employee
5. Exit
Enter choice: 1
Enter Emp No: 20
Enter Emp Name: kiran
```

```
Enter Salary: 90000
Employee inserted successfully.
===== AUSEMP TABLE MENU =====
1. Insert Employee
2. Update Employee Salary
3. Display Employees
4. Delete Employee
5. Exit
Enter choice: 3
----------------------------------
EmpNo   EmpName        Salary
----------------------------------
10      chakri         70000.0
20      kiran          90000.0
===== AUSEMP TABLE MENU =====
1. Insert Employee
2. Update Employee Salary
3. Display Employees
4. Delete Employee
5. Exit
Enter choice: 2
Enter Emp No to update salary: 10
Enter New Salary: 100000
Salary updated successfully.
===== AUSEMP TABLE MENU =====
1. Insert Employee
2. Update Employee Salary
3. Display Employees
4. Delete Employee
5. Exit
Enter choice: 3
----------------------------------
EmpNo   EmpName        Salary
----------------------------------
10      chakri         100000.0
```

```
20      kiran          90000.0
===== AUSEMP TABLE MENU =====
1. Insert Employee
2. Update Employee Salary
3. Display Employees
4. Delete Employee
5. Exit
Enter choice: 4
Enter Emp No to delete: 20
Employee deleted successfully.
===== AUSEMP TABLE MENU =====
1. Insert Employee
2. Update Employee Salary
3. Display Employees
4. Delete Employee
5. Exit
Enter choice: 3
----------------------------------
EmpNo   EmpName        Salary
----------------------------------
10      chakri         100000.0
===== AUSEMP TABLE MENU =====
1. Insert Employee
2. Update Employee Salary
3. Display Employees
4. Delete Employee
5. Exit
Enter choice: 5
Exiting program...
```